CTPIP

Installation and User Manual

Document control	
Document ID	CTPIP-UM-001
Project	Clinical Trial Protocol Intelligence Platform (CTPIP)
Version	1.1
Date	21 October 2026
Status	Released

Version	Date	Description of change
1.1	21 October 2026	Initial formal release.

Summary

- Installation of CTPIP on a new Windows computer, including the optional local AI (Ollama).
- Starting, using and stopping the application, with screenshots of every screen.
- Description of every project file and folder.
- Description of the dataset (187 breast-cancer protocols).

Table of Contents

1. Introduction 4

1.1 Purpose 4

1.2 Audience 4

1.3 System overview 4

1.4 Document conventions 5

2. System Requirements 5

3. Obtaining the Project 5

4. Installing the Python Environment 6

4.1 Install Python 6

4.2 Create the virtual environment and install packages 6

5. Configuring the Application 7

5.1 Create the settings file 7

5.2 Install and configure Neo4j Desktop 7

5.3 Settings reference 8

5.4 Local AI assistant (Ollama, optional) 9

6. Installing and Building the Frontend 9

7. Starting the Application 10

7.1 Quick start with start.bat 10

7.2 Manual start 10

7.3 Verify the start-up 11

7.4 Port already in use 11

8. Using the Application 11

8.1 Navigation 11

8.2 Dashboard 11

8.3 Protocols 12

8.4 Study a protocol 13

8.5 Ask AI and reading an AI answer 15

8.6 Search . . . 17
8.7 Compare . . . 18
8.8 Knowledge Graph . . . 19
8.9 History . . . 20
8.10 Settings . . . 21

9. Adding Input Data . . . 22

10. Stopping and Restarting . . . 22

11. Running Tests . . . 23

12. Troubleshooting . . . 24

13. Project File Reference . . . 25

13.1 Technology stack . . . 25
13.2 Root . . . 26
13.3 backend/ . . . 26
13.4 frontend/ . . . 29
13.5 data/ and docs/ . . . 31

14. Dataset Information . . . 31

14.1 Overview . . . 31
14.2 Dataset groups . . . 32
14.3 Content . . . 32
14.4 Output dataset . . . 33
14.5 Data quality notes . . . 34

Appendix A. Glossary . . . 35

1. Introduction

1.1 Purpose

- This manual explains how to install, start, use and stop the Clinical Trial Protocol Intelligence Platform (CTPIP).
- CTPIP reads clinical trial protocol PDFs and converts them into a searchable knowledge graph.
- A local AI assistant (optional) explains protocols and search results in plain language and answers questions, acting as a clinical-trial protocol expert.
- Every fact and every AI answer is linked to the page and the exact text of the source PDF.
- Everything runs locally on one computer: a FastAPI backend, a React web interface, a Neo4j graph database and, optionally, Ollama for the AI model.
- No cloud service or paid API key is required.

1.2 Audience

- Anyone who needs to understand what a protocol contains, including people without a medical or technical background.
- Professionals who search, compare and verify protocol details.
- Administrators who install and maintain the system.
- Technical terms are explained in Appendix A.

1.3 System overview

Component	Function	Address / port
Neo4j	Graph database that stores the knowledge graph.	7687
Backend (Python, FastAPI)	Reads PDFs, builds the graph, answers questions, runs the AI assistant and serves the web interface.	http://127.0.0.1:8000 (or next free port)
Web interface (React)	Screens used in the browser; served by the backend.	same as backend
Ingestion worker	Background process that reads uploaded PDFs in parallel.	internal
SQLite database	Stores documents, text passages, warnings, history, AI settings and saved explanations.	file in data/
Ollama (optional)	Runs the local AI model used for explanations and Ask AI.	11434

1.4 Document conventions

Convention	Meaning
`<project-folder>`	The folder that contains `start.bat`, `backend/` and `frontend/`. Replace it with the real location on your computer.
`monospaced text`	A file name, folder, command or value to be typed exactly.
NOTE	Additional information.
WARNING	An action that may cause data loss or a failure.

2. System Requirements

Item	Requirement
Operating system	Windows 10 or Windows 11 (the `start.bat` launcher is Windows-only).
Python	Version 3.11 or newer, with `pip`.
Node.js	Current LTS version, with `npm`.
Neo4j Desktop	Community or Enterprise edition. Includes the Java 21 runtime that Neo4j needs.
Ollama (optional)	Needed only for the local AI features. Free download from ollama.com.
Web browser	Chrome, Edge or Firefox on the same computer that runs CTPIP.
Disk space	Approximately 3 GB for the project and packages, plus 2 to 5 GB for an AI model.
Memory / graphics	8 GB RAM or more recommended. No GPU is required; with local AI, a graphics card with 4 GB or more makes answers much faster.
Network	Internet access only for the first installation of packages and the AI model download. OCR models are bundled.

3. Obtaining the Project

- Option A - Git: clone the repository and change into the project folder.

```
git clone <repository-url>
cd <project-folder>
```

- Option B - ZIP: extract the archive and open a terminal in the extracted folder.
- The project includes the 190 dataset PDFs (about 450 MB) and a `data/` folder with processed results (about 700 MB).
- Open Windows PowerShell and move to the project folder, then list its contents:

```
cd <project-folder>
dir
```

- The listing must show `start.bat`, `backend`, `frontend`, `data` and `docs`.
- If it does not, the terminal is in the wrong folder.
- Shortcut: in File Explorer open the project folder, click the address bar, type `cmd` and press Enter.

4. Installing the Python Environment

4.1 Install Python

1. Download Python 3.11 or newer from python.org.
2. Run the installer and tick **Add python.exe to PATH**.
3. Open a new terminal and check the version:

```
python --version
```

4.2 Create the virtual environment and install packages

- A virtual environment (`.venv`) is a private set of Python libraries used only by this project.
- Run the following in Windows PowerShell from the project folder:

```
py -3.11 -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r backend\requirements.txt
```

- If PowerShell blocks script activation, use Command Prompt instead:

```
py -3.11 -m venv .venv
.venv\Scripts\activate.bat
python -m pip install --upgrade pip
pip install -r backend\requirements.txt
```

- Keep the environment activated for later Python commands; re-activate it in every new terminal.
- If installation fails, confirm that the selected Python is 3.11 or newer.
- `start.bat` performs these steps automatically on its first run.
- macOS and Linux: the backend and frontend are cross-platform, but `start.bat` is not. Equivalent commands (not tested for this project): `python3.11 -m venv .venv`, `source .venv/bin/activate`, `pip install -r backend/requirements.txt`.

5. Configuring the Application

5.1 Create the settings file

- All settings are read from a file named `.env` in the project folder.
- Create it from the supplied example:

```
Copy-Item .env.example .env
```

WARNING: The `.env` file contains passwords. Do not commit it to source control and do not share it.

5.2 Install and configure Neo4j Desktop

1. Install and open Neo4j Desktop.
2. Create a local project and a **Local instance** (DBMS); set a password; start the instance once to confirm it works.
3. Note the Bolt address (normally `neo4j://127.0.0.1:7687`), the user name and the password.
4. Note the instance installation folder (it contains `bin`, `conf` and `data`).
5. Note the Java 21 runtime folder used by Neo4j Desktop (a folder named like `zulu21...`).
6. Edit `.env` with Notepad and set these values:

```
NEO4J_URI=neo4j://127.0.0.1:7687
NEO4J_USER=neo4j
NEO4J_PASSWORD=<password-set-in-Neo4j-Desktop>
NEO4J_DATABASE=ctpip
NEO4J_HOME=<path-to-neo4j-dbms-folder>
NEO4J_JAVA_HOME=<path-to-java-21-runtime>
```

- Use `NEO4J_DATABASE=neo4j` on Neo4j Community edition (only Enterprise supports an additional database named `ctpip`).
- If `NEO4J_HOME` is set, `start.bat` starts Neo4j automatically; otherwise start the instance manually before the backend.
- Restart the backend after any change to `.env`.

5.3 Settings reference

Setting	Default	Description
NEO4J_URI	neo4j://127.0.0.1:7687	Address of the graph database.
NEO4J_USER, NEO4J_PASSWORD	neo4j, (required)	Graph database login.
NEO4J_DATABASE	ctpip	Database name.
NEO4J_READ_USER, NEO4J_READ_PASSWORD	empty	Optional read-only account used for every question.
NEO4J_HOME, NEO4J_JAVA_HOME	empty	Folders that allow start.bat to launch Neo4j.
QUERY_TIMEOUT_S	15	Seconds before a question is stopped.
QUERY_RESULT_LIMIT	400	Maximum results per question.
INGEST_WORKERS	4	PDFs analysed in parallel.
OCR_ENABLED	true	Read scanned pages with OCR.
TABLE_DETECTION	true	Detect tables in PDFs.
MAX_UPLOAD_MB	200	Largest accepted PDF.
IMPORT_DIR	project folder	Folder scanned by Import dataset folder.
EXTRACTION_PROVIDER	rules	rules = built-in logic; openai_compatible = also use an AI model (output is verified against the text).
ANSWER_PROVIDER	none	none = evidence-only answers; openai_compatible = AI may reword them.
LLM_BASE_URL, LLM_MODEL, LLM_API_KEY	empty	Connection to the optional AI model.
API_TOKEN	empty	If set, every /api request must send header X-API-Token.
BACKEND_PORT	8000	Port used by start.bat (the next free port is used if it is taken).
AUTO_IMPORT_ON_EMPTY	true	On the first start, read every dataset PDF automatically.
AI_ENABLED	true	Switch the local AI features on or off.
AI_PROVIDER, OLLAMA_URL	ollama, http://127.0.0.1:11434	Type and address (host and port) of the local model server.
AI_MODEL	qwen2.5:3b	Model used for explanations and Ask AI.
AI_TEMPERATURE, AI_CONTEXT_TOKENS	0.2, 8192	Creativity of the answers and how much text the model reads at once.

5.4 Local AI assistant (Ollama, optional)

- The AI features (plain-language explanations, protocol Q&A;, Ask AI, AI toggles in search and compare) need a local model server.
- All other features work without it.

1. Download and install Ollama from ollama.com (Windows installer). It runs in the background on port 11434.
2. Run `start.bat`. If the model in `AI_MODEL` (default `qwen2.5:3b`, about 2 GB) is missing, the launcher asks whether to download it; press **Y**.
3. Alternatively open **Settings** in the application, type a model name under **Download a model** and press **Download**.
4. In **Settings** choose the model, press **Save settings** and then **Test the model**. The status shows **Ready**.

Model	Use
`qwen2.5:3b`	Default. Fast; fits a 4 GB graphics card.
`qwen2.5:7b`	Better writing and more reliable citations; needs about 6 to 8 GB of graphics memory (much slower on 4 GB cards).
`llama3.1:8b`	Good general model; about 8 GB of graphics memory.

- Settings in `.env` (`AI_ENABLED`, `AI_PROVIDER`, `OLLAMA_URL`, `AI_MODEL`, `AI_TEMPERATURE`, `AI_CONTEXT_TOKENS`) are defaults; changes made on the Settings page override them.
- Any OpenAI-compatible server (LM Studio, llama.cpp server, vLLM) can be used instead: choose **OpenAI-compatible** on the Settings page and enter its address.
- Protocol text is sent only to the configured local server; nothing leaves the computer.

6. Installing and Building the Frontend

- The frontend is the web interface. It must be built once; the backend then serves it on the same port.
- Run in Windows PowerShell:

```
cd frontend
npm install
npm run build
```

- The build creates `frontend/dist`. `start.bat` runs these commands automatically when the folder is missing.
- Developer mode (optional): `npm run dev` starts a Vite server on `http://127.0.0.1:5173` that forwards `/api` requests to the backend on port 8000.

7. Starting the Application

7.1 Quick start with start.bat

- In File Explorer double-click `start.bat`, or run it from PowerShell in the project folder:

```
.\start.bat
```

- Windows stay open for the launcher, Neo4j, the backend and (if started) Ollama.
- Do not close the Neo4j or backend window while using the application.
- Use `start.bat dev` to start the Vite development server instead of the built interface.

Step	Action performed by start.bat	Expected output
0	Reads `.env` (Neo4j folders, `BACKEND_PORT`, `AI_ENABLED`, `AI_MODEL`).	If the file is missing: `[!] .env missing`.
1	Checks Neo4j on port 7687; starts it from `NEO4J_HOME` if needed (waits up to 3 minutes).	`[*] Neo4j already running` or `[*] Starting Neo4j from ...`
2	Local AI: starts Ollama if installed and not running; offers to download the AI model if missing.	`[*] Local AI ready: qwen2.5:3b`, or a hint that Ollama is not installed (optional).
3	Creates `.venv` and installs `backend\requirements.txt` (first run only).	Package download messages.
4	Finds a free port, starting at `BACKEND_PORT` (8000).	If 8000 is taken: `[i] Port 8000 is used by another program - CTPIP will use port 8001 instead.`
5	Runs `npm install` and `npm run build` if needed (first run only).	npm messages.
6	Starts the backend, waits for `/api/health` and opens the browser. On the very first start the dataset PDFs are read automatically.	`[*] CTPIP running at http://127.0.0.1:8000`

NOTE: First start on a new computer: the 190 dataset PDFs are read into the database and the knowledge graph automatically (about 30 to 60 minutes). The Protocols page shows the progress; the application can be used meanwhile.

7.2 Manual start

- Start the Neo4j instance (Neo4j Desktop, or `neo4j.bat console`) and, for AI, Ollama.
- In a terminal at the project folder activate the environment and start the backend:

```
& .venv\Scripts\Activate.ps1
cd backend
uvicorn app.main:app --host 127.0.0.1 --port 8000
```

- Keep the terminal open. Open `http://127.0.0.1:8000` in the browser.

7.3 Verify the start-up

- Open `http://127.0.0.1:8000/api/health`. It must answer with `"status":"ok"` and `"graph":"online"`.
- The bottom of the left menu shows **Database online** and **Local AI** with the model name when AI is ready.
- Interactive API documentation is available at `http://127.0.0.1:8000/docs`.

7.4 Port already in use

- start.bat automatically uses the next free port and prints the address it chose.
- For a manual start choose another port yourself, for example `--port 8010`.

8. Using the Application

8.1 Navigation

Menu item	Purpose
Dashboard	Totals, quick actions, recent activity and data quality.
Protocols	Add protocols and open any protocol to study it.
Ask AI	Ask the local AI about one protocol or all protocols.
Search	Smart search over verified facts, or exact-wording text search.
Compare	Compare 2 to 4 protocols field by field.
Knowledge Graph	Explore the connections visually.
History	Past questions and the activity log.
Settings	Connect and choose the local AI model.

- Every querying area has an **Explain with local AI** switch. It is remembered in the browser. When it is on, results are also explained in plain language.

8.2 Dashboard

- Four tiles: protocols ready, facts found (and how many are checked against the text), drugs/tests/outcomes, and the local AI status.
- Quick actions: Study a protocol, Ask AI, Search, Compare.
- Recent questions and recent activity.

- Data quality (bottom): summary, drug names awaiting a human decision (merge or keep separate), differences between sources, reading warnings.

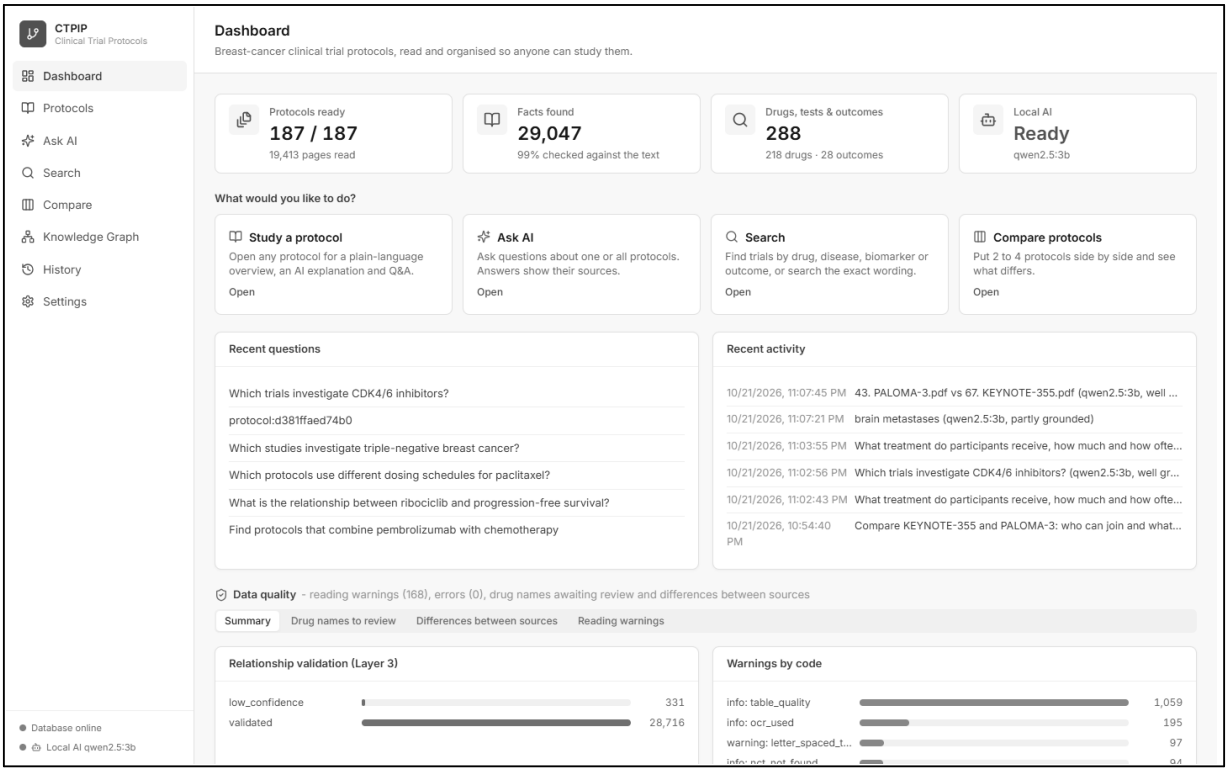


Figure 1: Dashboard

8.3 Protocols

1. Open **Protocols**.
2. Add PDFs: drag them onto the dashed area or click **Choose PDFs**; or press **Load dataset folder** to add every PDF in the dataset folders.
3. Wait until the status is `ready` (about 10 seconds for a short file, 20 to 60 seconds for a 150-page protocol).
4. Press **Study** to open a protocol.

- Search box: find by name, drug, trial number or sponsor; the drop-down filters by status.
- Show technical columns: version, type, entities, facts and warnings.
- A failed protocol shows a retry button; the bin button removes a protocol and all its data (confirmation required).

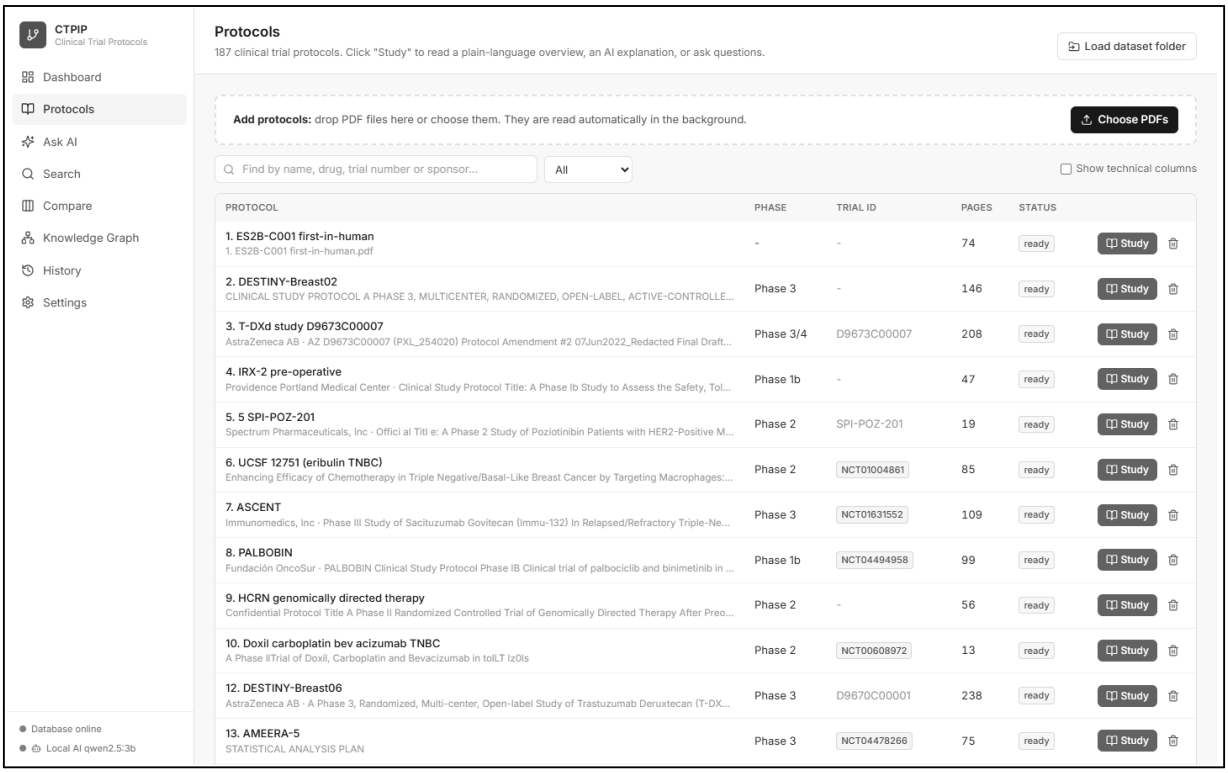


Figure 2: Protocols list

8.4 Study a protocol

Tab	What it shows
Overview (plain language)	One-sentence summary; key facts with plain explanations (phase, design, disease, setting, sponsor, trial ID); what participants receive; who can and cannot take part; what is measured; safety; tests; follow-up; the treatments explained; words in the protocol explained. Every item has a page link to the original PDF. Works without AI.
AI explanation	A complete plain-language explanation written by the local AI, with citation chips. Saved after the first run; **Regenerate** writes a new one.
Ask questions	Chat about this protocol only (see 8.5).
Original document	Detected table of contents and the text of each section, with **View on page**.
Technical details	Processing status, tables, entities, warnings and Reprocess.

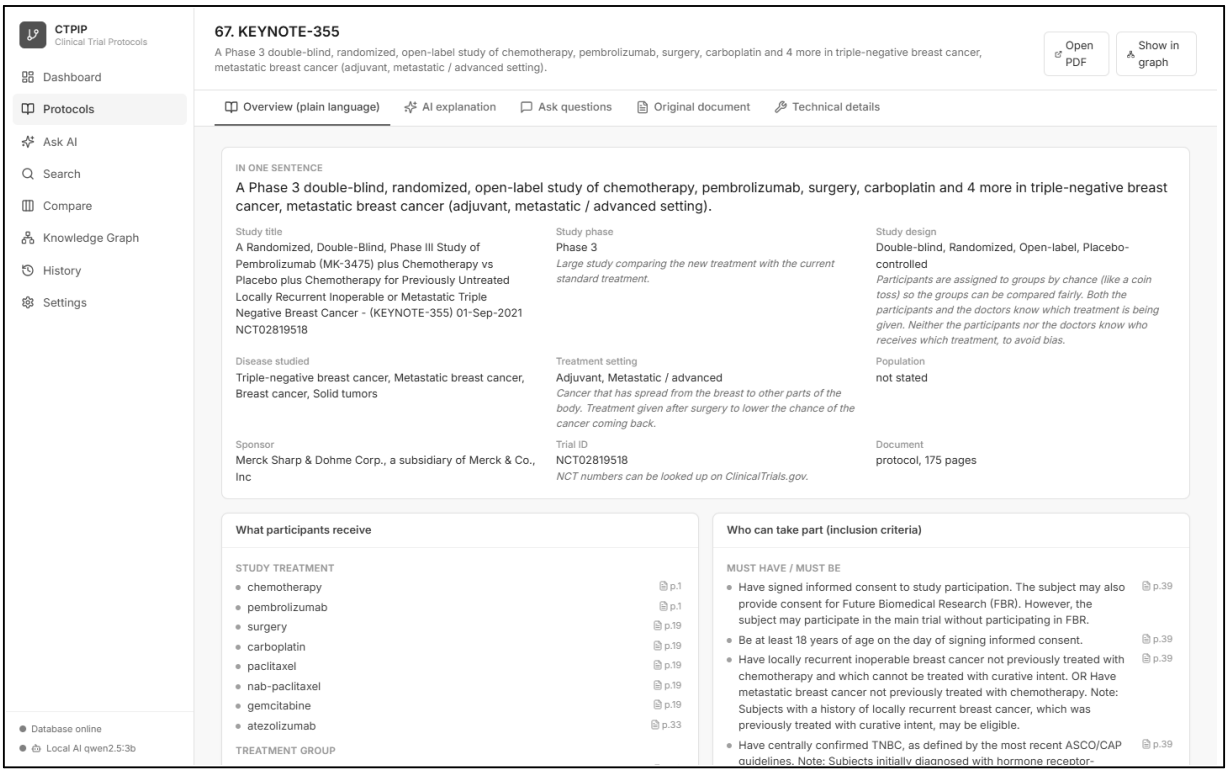


Figure 3: Protocol overview in plain language

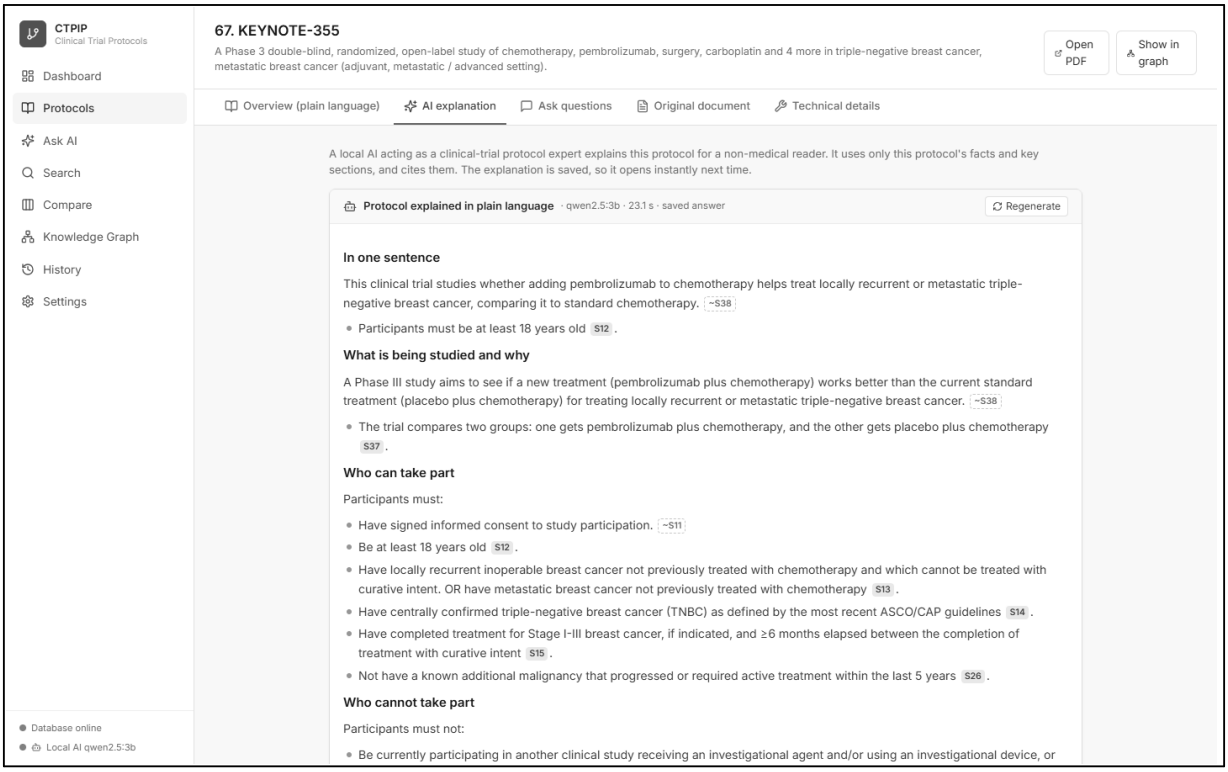


Figure 4: AI explanation of a protocol with citations

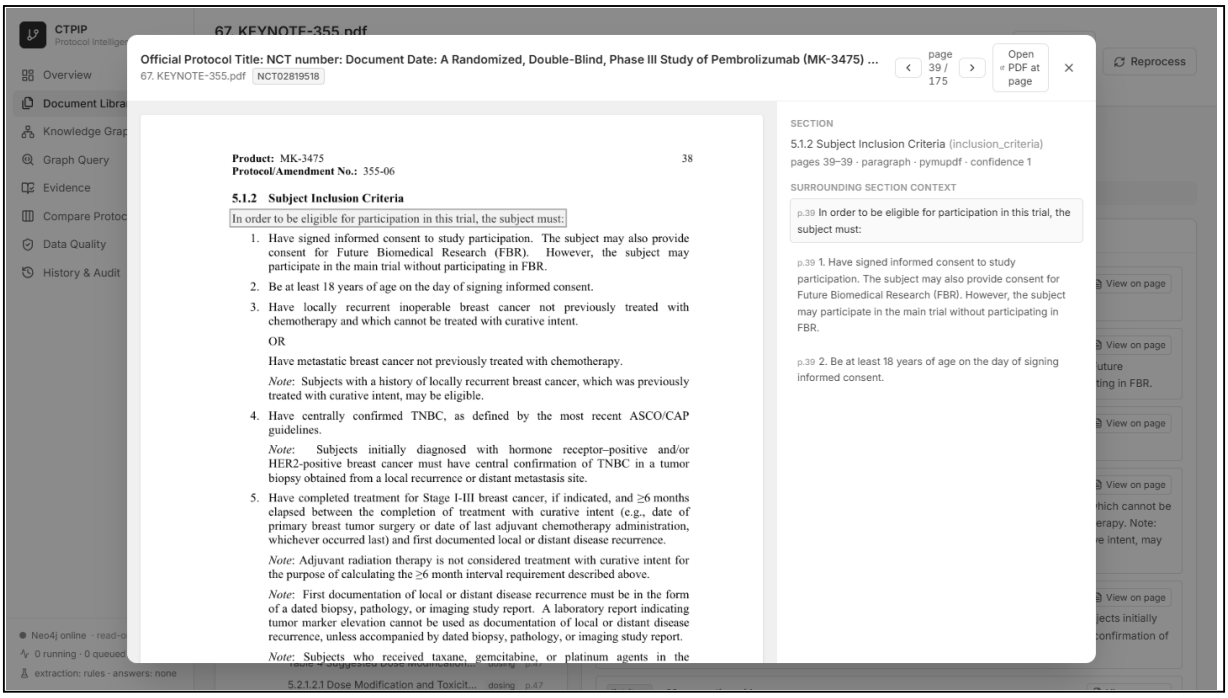


Figure 5: Source viewer: the original PDF page with the passage highlighted

8.5 Ask AI and reading an AI answer

1. Open **Ask AI** (or the **Ask questions** tab of a protocol).
2. Choose **All protocols** or pick one protocol on the left.
3. Choose the answer style: **Quick answer** (searches and answers in seconds) or **Research agent** (plans several searches and comparisons; slower).
4. Type a question or click a suggestion, then press **Ask**. Follow-up questions keep the conversation context.

Element of an answer	Meaning
Blue chip `S3`	The AI cited source 3 for this statement. Click it to highlight the source.
Dashed chip `~S3`	The AI did not cite a source; the system matched the statement to source 3 automatically by comparing words.
Explainability check	How many statements cite a source, how many were matched automatically, numbers checked against the sources (numbers not found are listed for you to verify), and citations to sources that do not exist. Verdict: well, partly or weakly grounded.
Sources given to the AI	Every passage or fact the AI was allowed to use, why it was chosen (matched words, relevant section, graph fact), and View page.
How the research agent worked	Agent mode only: each step with its reasoning, the tool used (find protocols, protocol overview, text search, graph search, compare) and what it found.

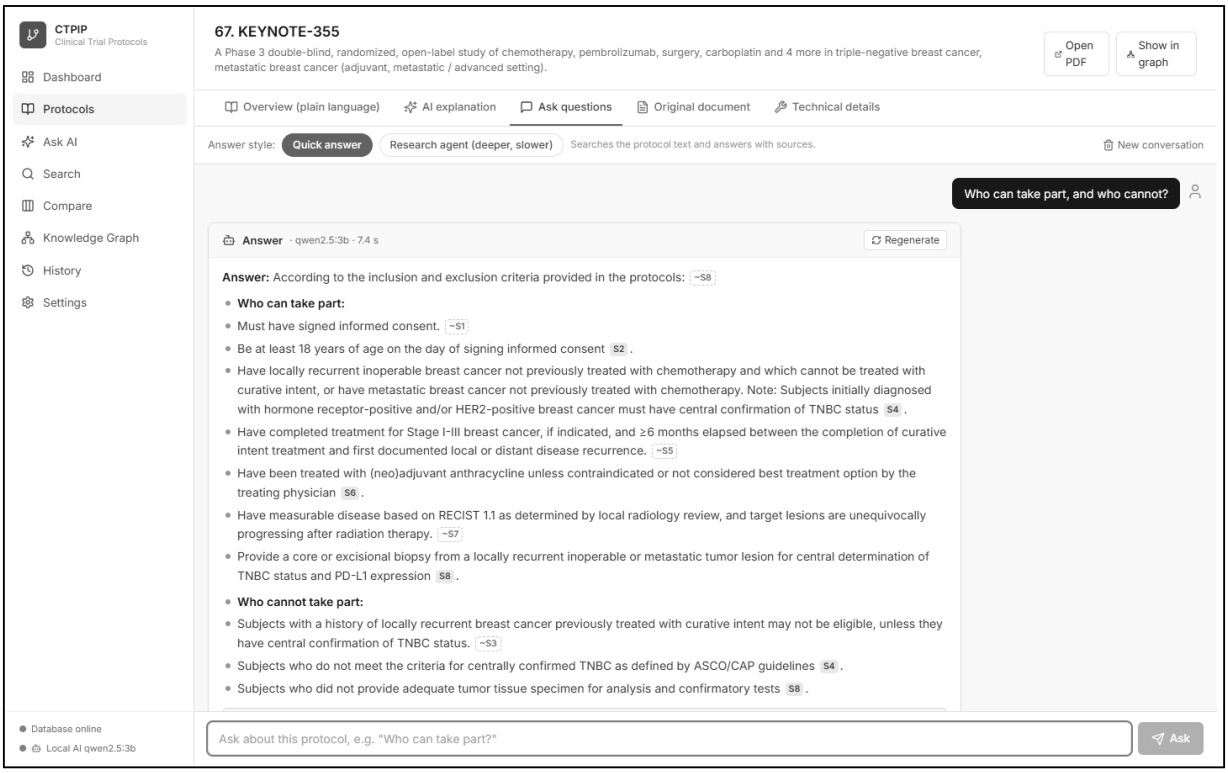


Figure 6: Asking a question about one protocol

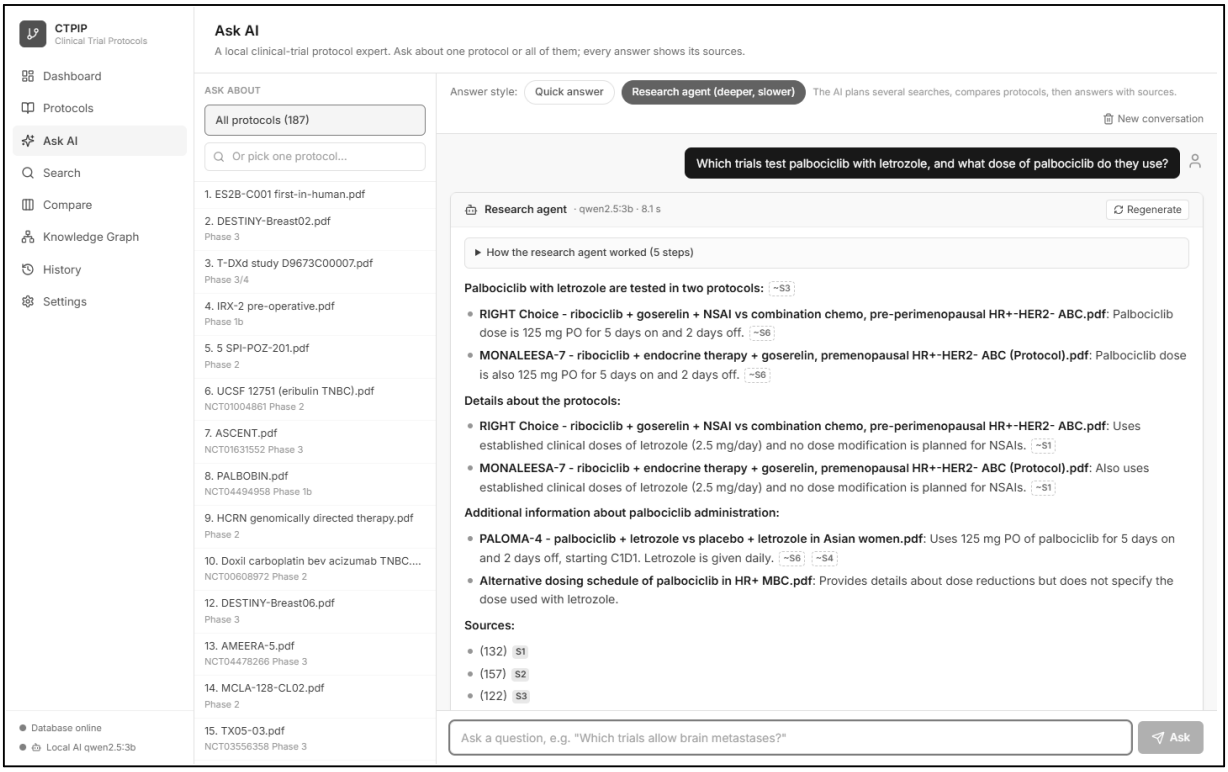


Figure 7: Ask AI in research-agent mode across all protocols

NOTE: The AI explains what the documents say; it is not medical advice. Small models occasionally phrase things imprecisely - the citations, page links and the explainability check are there so that every statement can be verified.

8.6 Search

- **Smart search**: type a question (or click an example) and press **Run query**. The answer uses verified graph facts; the tabs show the answer tree, the graph, reasoning paths and the query plan.
- **Text search**: finds exact words in every protocol page, optionally limited to a section type or one protocol.
- With **Explain with local AI** switched on, an AI explanation of the results appears with sources and the explainability check.

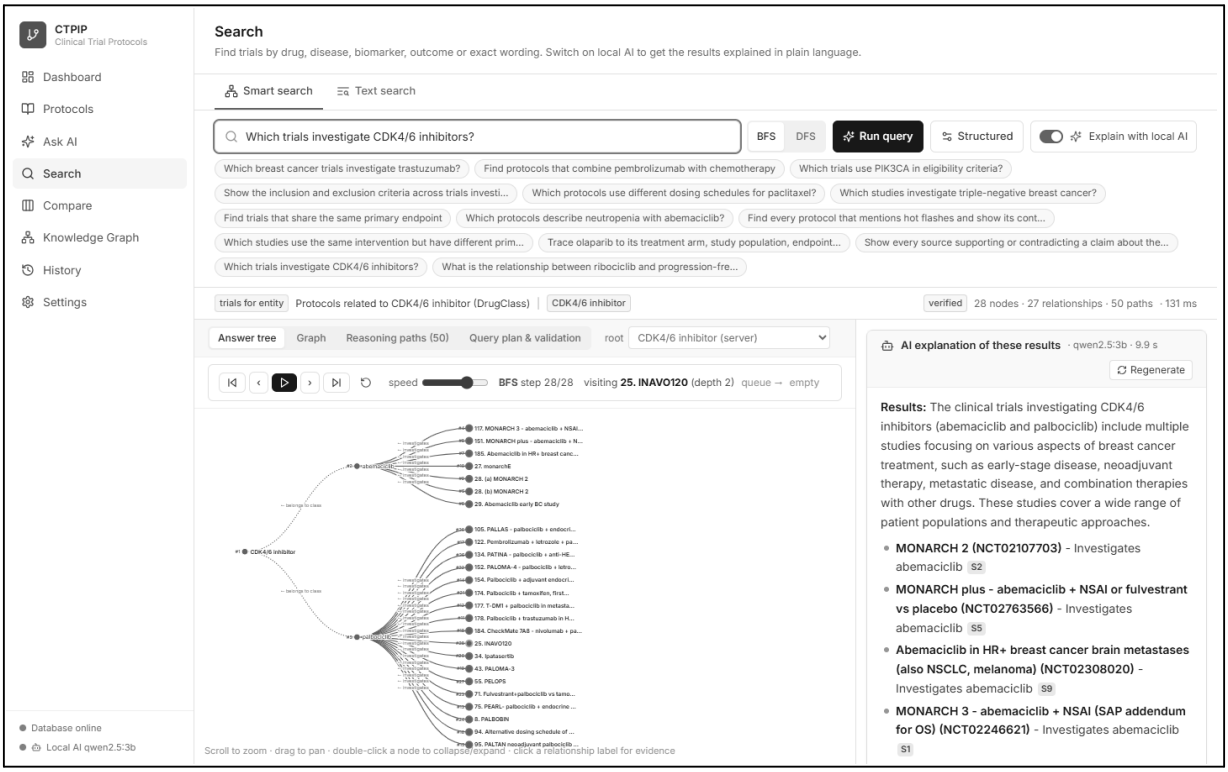


Figure 8: Smart search with AI explanation

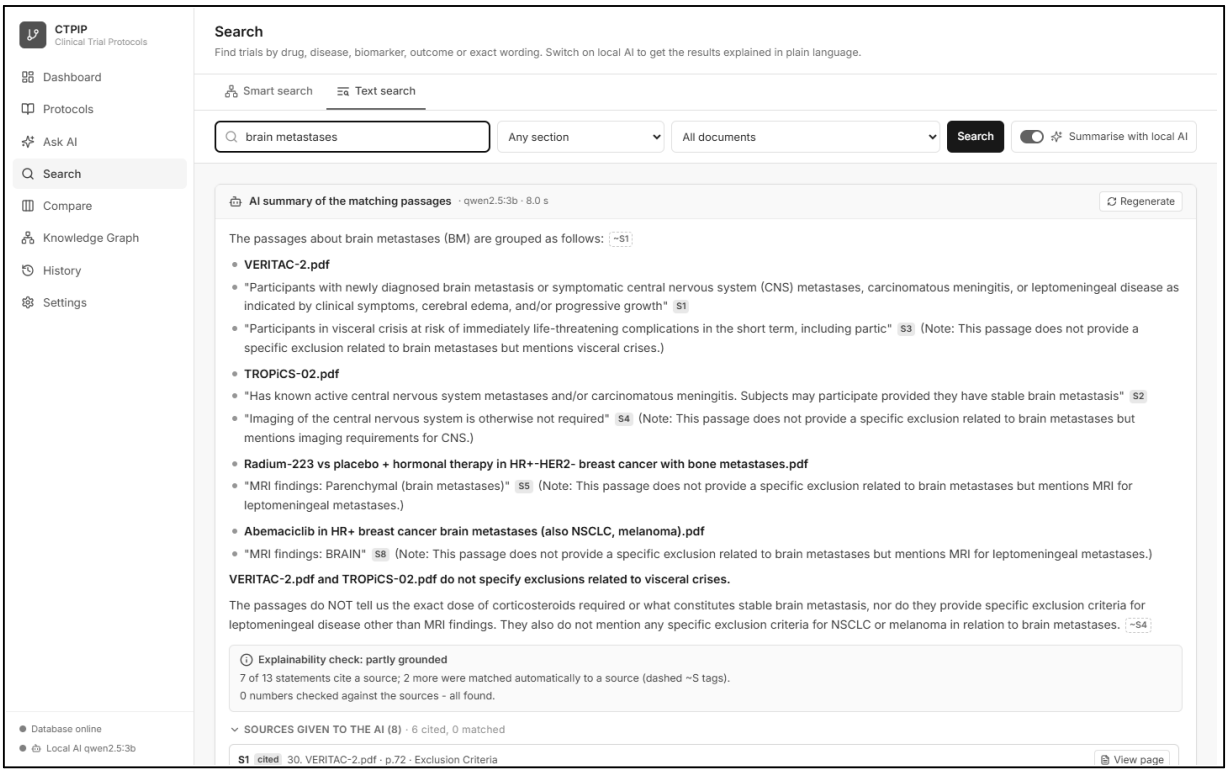


Figure 9: Text search with AI summary

8.7 Compare

1. Open **Compare** and tick 2 to 4 protocols.
2. Optionally switch on **Explain the differences with AI**.
3. Press **Compare N protocols**.

- Each row (phase, design, disease, setting, drugs, arms, doses, criteria, endpoints, safety) is labelled same, similar, different, conflicting or unavailable.
- Unavailable means not found in the document; it does not mean excluded.
- Every value links to its source passage.

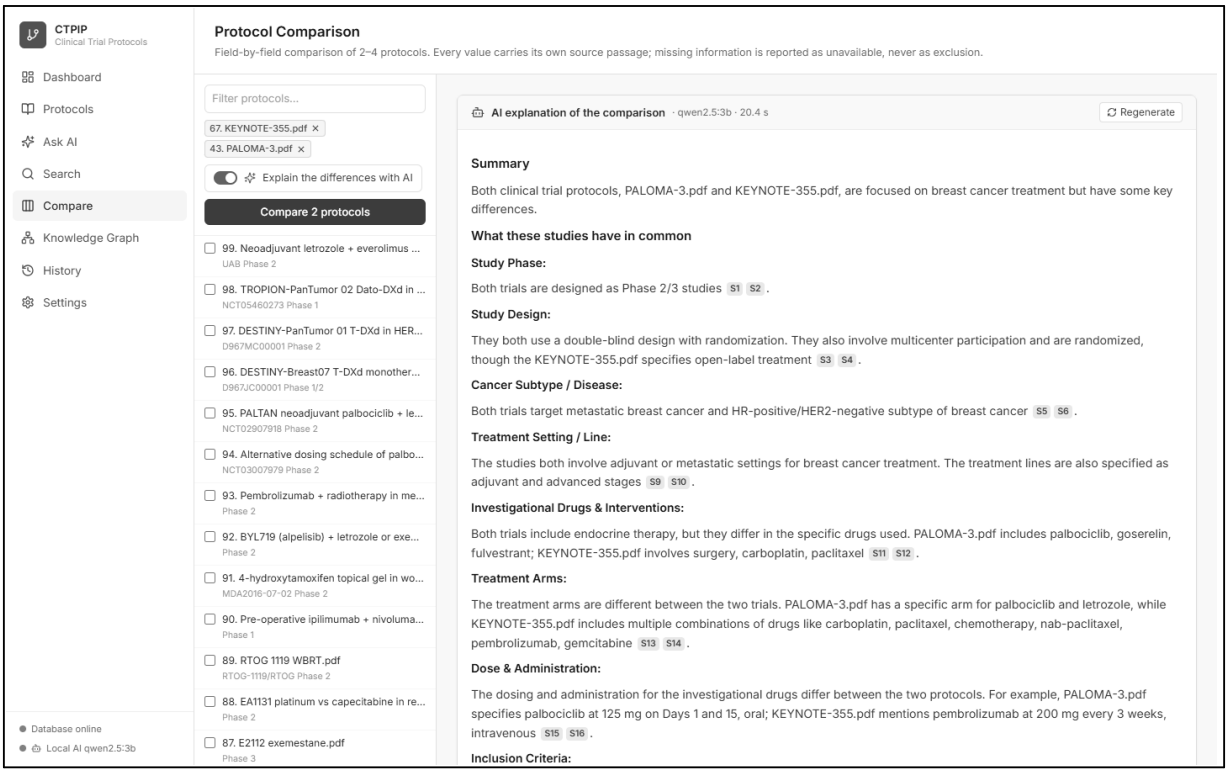


Figure 10: Comparison with AI explanation

8.8 Knowledge Graph

- Press **Load overview**; click a dot or line to see details and sources; double-click to expand neighbours; **Traverse** replays BFS or DFS step by step.

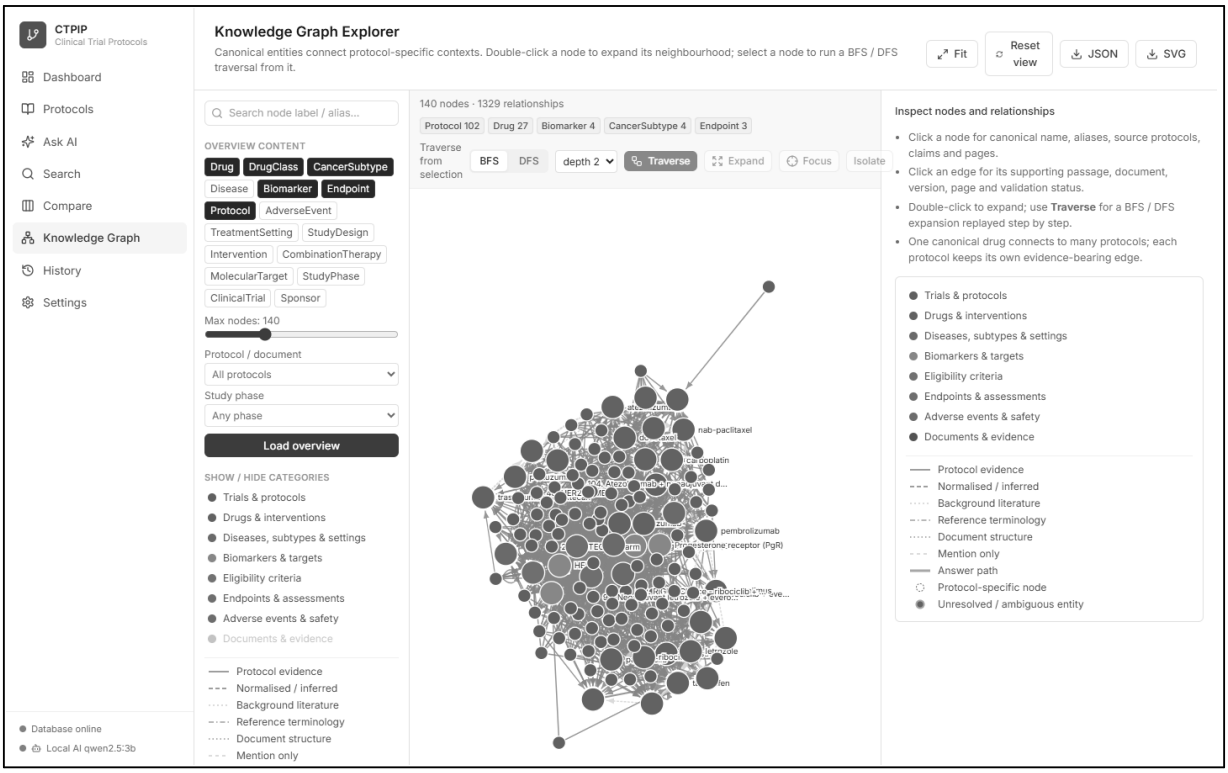


Figure 11: Knowledge Graph explorer

8.9 History

- Query history with plan, evidence and validation; the activity log also records AI answers (model and grounding verdict).

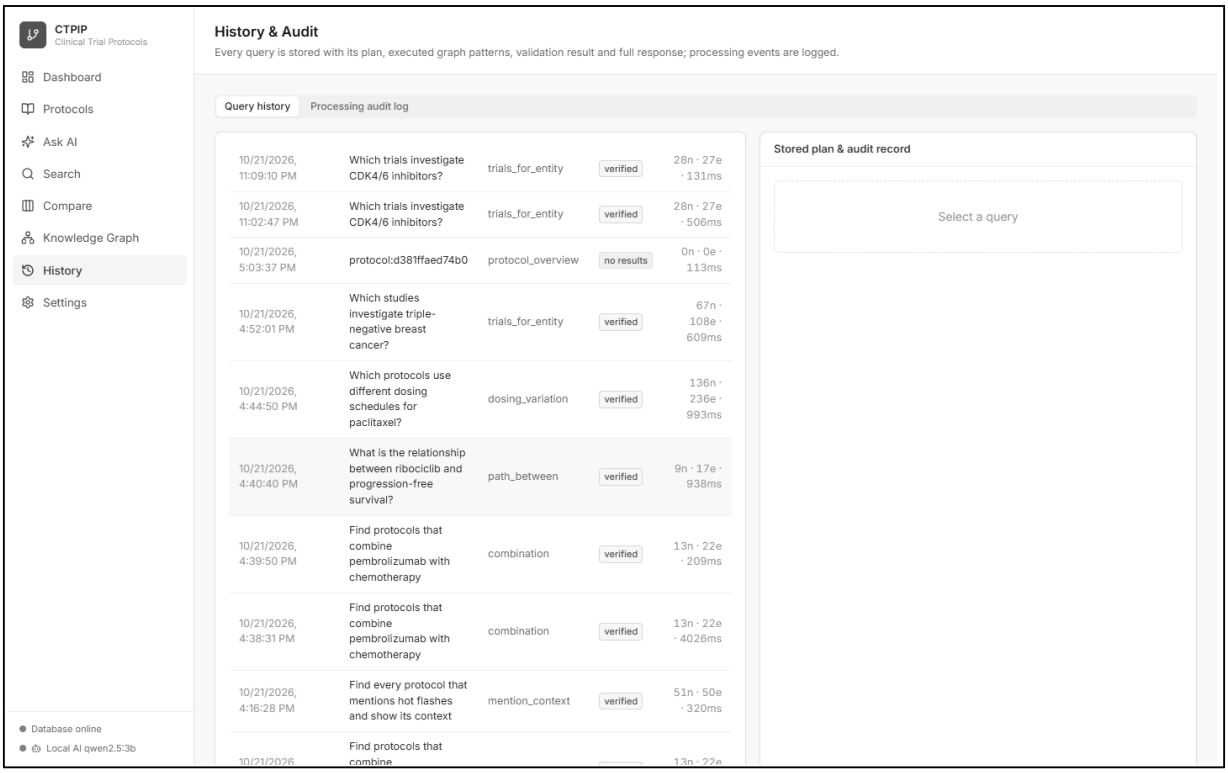


Figure 12: History and audit log

8.10 Settings

- Local AI status, server type and address (port), model, temperature, context size, model test and model download (see 5.4).

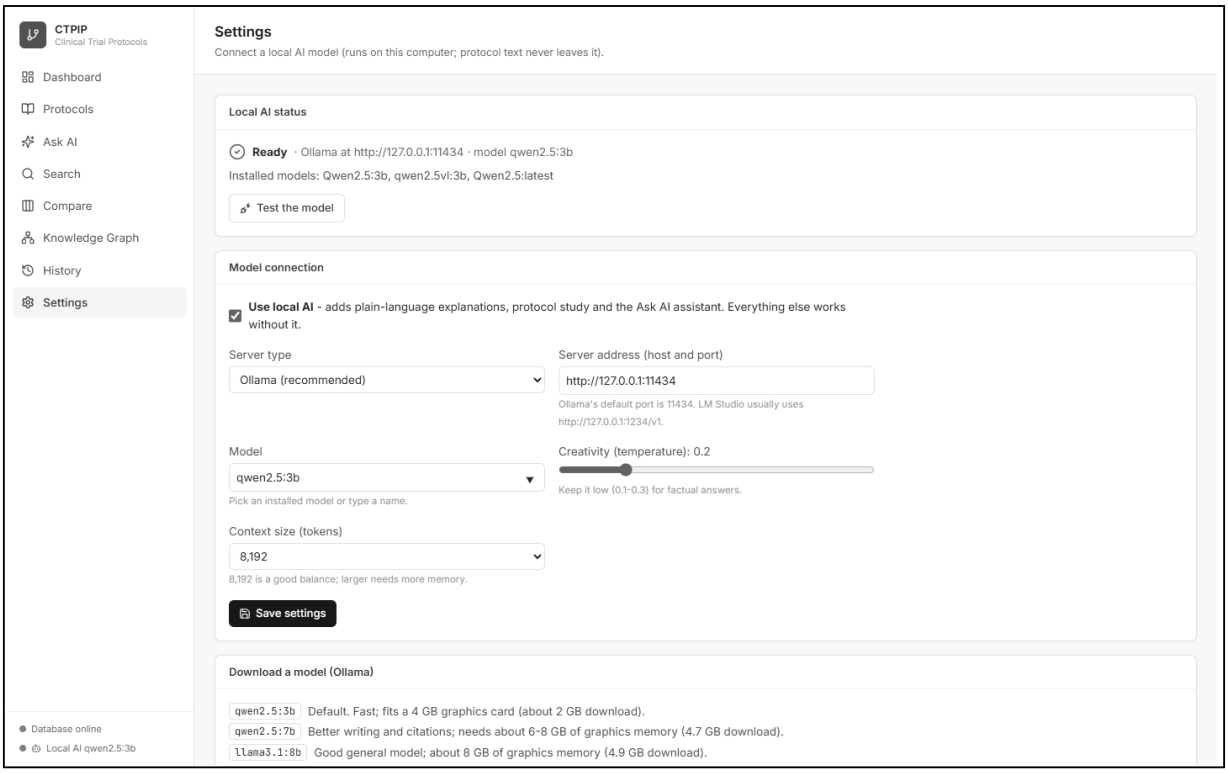


Figure 13: Settings: local AI configuration

9. Adding Input Data

- Accepted input: real PDF files that are not password-protected and are smaller than `MAX_UPLOAD_MB`.
- Upload: use Protocols as described in section 8.3.
- Load dataset folder: queues every PDF under `IMPORT_DIR` (default: the project folder), including sub-folders. The folders `docs/`, `backend/`, `frontend/`, `data/`, `.venv` and `node_modules` are skipped.
- First start on a new computer: when no documents are stored yet, the dataset is loaded automatically (`AUTO_IMPORT_ON_EMPTY=true`).
- Duplicate files are detected by content (SHA-256), even when renamed, and skipped.
- Scanned pages are read with built-in OCR.
- A practice file is provided: `docs/demo/Sample_Protocol_SAMPLE-PEMBRO-001.pdf` (a made-up protocol, not a real trial).

10. Stopping and Restarting

- Click the backend window and press **Ctrl+C**
- Click the Neo4j window, press **Ctrl+C** (or stop the instance in Neo4j Desktop) and wait a few seconds.
- In developer mode also press Ctrl+C in the frontend terminal.
- Restart by running `start.bat` again.

- Documents, graph and history are kept (the `data/` folder and Neo4j).
- If the computer stops during processing, unfinished documents resume automatically from their last completed stage.

11. Running Tests

- The test runner `pytest` is installed with the backend requirements.
- Run from the backend folder with the environment active:

```
cd backend
python -m pytest
```

- `backend/tests/test_ai_explainability.py` checks the grounding check, the glossary and the search-term handling; it needs no database or AI model.

12. Troubleshooting

Symptom	Cause / action
`[!] .env missing`	Create the file: `Copy-Item .env.example .env` and set the Neo4j password.
`[!] Neo4j is not running and NEO4J_HOME is not set`	Set `NEO4J_HOME` and `NEO4J_JAVA_HOME` in `.env`, or start Neo4j manually.
`[!] Neo4j did not start`	Check the two paths; close any other Neo4j using the same files; read the Neo4j window.
`[!] Python 3.11+ required`	Install Python 3.11 or newer with Add to PATH ticked; open a new terminal.
Neo4j unavailable (status dot not filled)	Start the Neo4j instance; verify `NEO4J_URI`, `NEO4J_PASSWORD`, `NEO4J_DATABASE`; restart the backend.
Port 8000 or 5173 already in use	start.bat picks the next free port automatically; for a manual start use `--port 8010`.
Browser: site cannot be reached	Backend not running or still starting; wait for `CTPIP ready` in the backend window.
`Frontend not built` (503)	Run `npm install` and `npm run build` in `frontend`.
`.venv` cannot be activated in PowerShell	Use `.venv\Scripts\activate.bat` in Command Prompt.
Upload rejected	File is not a PDF, is encrypted, is corrupt or exceeds `MAX_UPLOAD_MB`.
Upload duplicate	The same content is already loaded.
Document status failed	Open the document to read the error and press Retry. After 3 automatic attempts it stays failed. See `data/logs/server.log`.
Query returns unsupported / no results	Use drug, disease, biomarker or endpoint names present in the data; try an example question; use Evidence search.
First question is slow	The engine is warming up (about 10 seconds).
`Run evaluation` fails	`api/evaluation.py` imports `app.evaluation.runner`, which is not present in the project.
Local AI: not ready / server not reachable	Install or start Ollama (`ollama serve`); check the address on the Settings page (default `http://127.0.0.1:11434`).
Local AI: model is not installed	Download it on the Settings page or run `ollama pull qwen2.5:3b`; then choose it and press Save.
AI answers are slow	The first answer loads the model (10 to 20 seconds). Use `qwen2.5:3b` on small graphics cards; larger models run partly on the CPU.
AI answer marked partly or weakly grounded	Read the sources and page links; rephrase the question with drug, disease or test names; try Research agent mode or a larger model.

Symptom	Cause / action
Ollama is not installed message in start.bat	Optional: install Ollama from ollama.com to enable AI features; everything else works without it.

13. Project File Reference

- This section describes every file and directory in the project.
- Analogy: backend = machinery, frontend = control panel, data = warehouse, numbered PDF folders = raw materials.

13.1 Technology stack

Layer	Technology	Purpose
Frontend	React 18, TypeScript 5.6	User interface.
Frontend	Vite 5	Build tool and development server.
Frontend	React Router 6	Page navigation.
Frontend	TanStack Query 5	Data fetching and caching.
Frontend	D3.js 7	Graph drawing.
Frontend	Tailwind CSS 3.4, lucide-react, clsx	Styling and icons.
Backend	FastAPI, Uvicorn	HTTP API (about 35 endpoints) and server.
Backend	Pydantic v2, pydantic-settings	Validation and settings.
Backend	python-multipart, httpx, pytest	Uploads, optional AI calls, tests.
Graph	Neo4j 5 and Python driver 5.28+	Persistent knowledge graph (Cypher).
Storage	SQLite (WAL, FTS5)	Metadata and full-text search.
Storage	gzip JSON files	Per-document checkpoints.
PDF	PyMuPDF	Text, positions, tables, page images.
PDF	RapidOCR (ONNX Runtime)	Offline OCR of scanned pages.
Language	Curated breast-oncology lexicon, regular expressions	Entity and fact extraction.
Infrastructure	start.bat, Java 21 (via Neo4j Desktop)	Launcher and Neo4j runtime.

13.2 Root

Item	Description
`start.bat`	Launcher: starts Neo4j, prepares Python and frontend, starts the backend and opens the browser.
`.env`, `.env.example`	Local settings and the template to create them. Never commit `.env`.
`.gitignore`	Excludes `.env`, `.venv/`, `node_modules/` and caches from source control.
`.venv/`	Private Python libraries; created automatically.
`backend/`	The engine (Python code).
`frontend/`	The web interface.
`data/`	SQLite database, stored PDFs, checkpoints and logs.
`docs/`	Documentation, dataset catalog, slides and demo video.
`scripts/`	Empty; reserved for maintenance scripts.
`1-19/`, `20-38/`, `39-56/`, `57-100/`, `101-185/`	The dataset: 190 protocol PDFs (187 unique). See section 14.
`README.md`	Project overview and quick start shown on GitHub.
`.gitattributes`	Keeps Windows line endings for .bat files and marks PDFs, video and slides as binary.

13.3 backend/

File	Description
`requirements.txt`	Python package list installed by start.bat.
`app/main.py`	Entry point: prepares databases, starts the worker, serves the interface, optional API-token check, `/api/health`.
`app/config.py`	Reads `.env` and provides settings with defaults (including AI and first-start import).
`tests/test_ai_explainability.py`	Unit tests for the AI explainability helpers.
`__init__.py` files	Empty markers that make folders Python packages.

backend/app/api/

File	Description
`documents.py`	Documents: upload, import, list, details, sections, chunks, tables, entities, PDF, page image, retry, reprocess, delete, job queue.
`graph.py`	Graph: statistics, schema, overview, expand node (BFS/DFS), paths, entity search and detail.
`query.py`	Questions, evidence search, provenance, compare, quality, history, audit, dashboard numbers.
`evaluation.py`	Run and list evaluations (helper `runner.py` absent).
`assistant.py`	Local AI endpoints: status, settings, test, model download, protocol brief, protocol explanation, chat (RAG or agent), AI explanations of search, text search and comparison. Answers stream to the browser with sources and the explainability check.

backend/app/core/ and resolution/

File	Description
`core/metadb.py`	Defines the SQLite tables and read/write helpers.
`resolution/resolver.py`	Decides whether an unknown drug-like term is new, a known drug or needs human review (84% similarity threshold); never auto-merges look-alikes.

backend/app/ingest/

File	Description
`pipeline.py`	Per-document stages: register, analyse, store, resolve, write graph, delete.
`worker.py`	Job queue manager: up to 4 parallel analysis processes, 3 attempts per job, resume after crash.
`pdf_extract.py`	Reads text, fonts, positions, tables; OCR for scanned pages.
`structure.py`	Detects headings and classifies sections.
`chunker.py`	Splits sections into evidence passages with page and position.

backend/app/llm/ (local AI assistant)

File	Description
`settings.py`	AI settings (enabled, server, model, temperature, context size): defaults from .env, overrides saved from the Settings page; cache table for protocol explanations.
`client.py`	Talks to the local model server: Ollama native API or any OpenAI-compatible server; streaming answers, model list, model download, test.
`prompts.py`	Instructions for the clinical-trial protocol expert persona and for each task (explain protocol, chat, explain search, compare, agent planning).
`retrieval.py`	Finds the passages for a question (full-text search with synonym expansion and section-aware ranking) and validated graph facts; records why each source was chosen.
`protocol_brief.py`	Plain-language structure of one protocol from its graph facts and key sections (the Overview tab); also the grounding material for the AI explanation.
`glossary.py`	Plain-language explanations of medical terms and drug classes.
`agent.py`	Research agent: the model plans tool calls (find protocols, overview, text search, graph search, compare) and every step is reported.
`grounding.py`	Explainability check: cited sources, automatic matching of uncited statements, invalid citations, numbers not found in the sources.

backend/app/extraction/

File	Description
`lexicon.py`	Dictionary of breast-oncology terms and synonyms.
`entities.py`	Finds dictionary terms, learns document abbreviations, reads title-page data.
`relations.py`	Builds claims (facts) with quote, page and confidence.
`providers.py`	Rule-based extractor or optional AI model with output verification.

backend/app/graph/ and query/

File	Description
`graph/schema.py`	40 node types, 42 relationship types and allowed connections.
`graph/writer.py`	Writes a document's entities and claims to Neo4j; removes them on delete.
`graph/neo4j_client.py`	Neo4j connections: writer account and read-only transactions with timeout.
`query/planner.py`	Detects intent and selects safe query patterns.
`query/linker.py`	Links question words to graph entities.
`query/templates.py`	The only database queries allowed: pre-written, read-only, parameterised.
`query/engine.py`	Runs execution, ranking, validation and audit.
`query/traversal.py`	BFS/DFS answer tree and replay.
`query/answer.py`	Evidence-only answer text with support labels.
`query/compare.py`	Field-by-field comparison.
`query/serialize.py`	Converts graph objects to JSON.

13.4 frontend/

File	Description
`vite.config.ts`	Build settings; dev server forwards `/api` to port 8000.
`package.json`, `package-lock.json`	Web library list and exact versions.
`index.html`	Root HTML page.
`tsconfig.json`, `tsconfig.tsbuildinfo`	TypeScript settings and cache.
`tailwind.config.js`, `postcss.config.js`	Styling settings.
`dist/`	Built interface served by the backend.

frontend/src/

File	Description
`main.tsx`	Starts the application.
`App.tsx`	Left menu, database and AI status, and routes.
`index.css`	Global styles.
`lib/api.ts`	Backend client and data types.
`lib/ai.ts`	Streaming client for AI answers, AI status, and the remembered AI switches.
`lib/graphStyle.ts`	Colours and line styles.
`lib/traverse.ts`	BFS/DFS logic.
`pages/Dashboard.tsx`	Dashboard with data quality.
`pages/Documents.tsx`	Protocols list.
`pages/DocumentDetail.tsx`	Study a protocol (five tabs).
`pages/Assistant.tsx`	Ask AI.
`pages/Search.tsx`	Search page with Smart search and Text search tabs.
`pages/QueryWorkspace.tsx`	Smart search (graph query) with AI toggle.
`pages/EvidenceExplorer.tsx`	Text search with AI toggle.
`pages/Compare.tsx`	Compare with AI toggle.
`pages/GraphExplorer.tsx`	Knowledge Graph.
`pages/Quality.tsx`	Data-quality panel shown on the Dashboard.
`pages/History.tsx`	History and activity log.
`pages/Settings.tsx`	Local AI settings, test and model download.
`components/AiAnswer.tsx`	AI answer panel: citations, sources, explainability check, agent steps; the AI switch.
`components/ChatPanel.tsx`	Chat with the AI (quick answer or research agent).
`components/Md.tsx`	Formats AI answers and makes citation chips clickable.
`components/ui.tsx`	Buttons, cards, badges, tabs.
`components/GraphCanvas.tsx`	D3 graph drawing.
`components/DetailPanel.tsx`	Selection details panel.
`components/EvidenceCard.tsx`	One evidence passage.
`components/SourceViewer.tsx`	PDF page viewer with highlight.
`components/TraversalPlayer.tsx`	BFS/DFS replay controls.

File	Description
components/TreeView.tsx	Answer tree.
components/Legend.tsx	Graph legend.

13.5 data/ and docs/

Item	Description
data/ctpip.sqlite3 (+ -wal, -shm)	Metadata database; the -wal and -shm files are temporary companions.
data/pdfs/	Stored copy of every loaded PDF.
data/artifacts/<doc>/extraction.json.gz	Checkpoint 1: raw PDF reading result.
data/artifacts/<doc>/analysis.json.gz	Checkpoint 2: sections, chunks, entities, claims.
data/logs/server.log	Backend log.
docs/01_User_Manual.pdf	This document.
docs/00_Project_Overview.pdf	Architecture and technology overview.
docs/02_Dataset_Information.pdf	Dataset description.
docs/03_Pipeline_Explanation.pdf	Pipelines with worked example.
docs/04_Module_Architecture.pptx	Slides: modules.
docs/05_Pipeline.pptx	Slides: pipelines.
docs/Dataset_Catalog.csv	One row per document.
docs/CTPIP_Demo_Video.mp4	Silent walkthrough (about 4 minutes).
docs/How_To_Run.pdf	One-page quick reference.
docs/images/	Screenshots of the application.
docs/demo/	Made-up sample protocol for practice.

14. Dataset Information

14.1 Overview

- The input documents are clinical trial protocol PDFs about breast cancer, written between 2006 and 2025.
- A protocol is the master plan of a study: purpose, eligibility, treatments and doses, safety monitoring, measurements and analysis.

- Sources: pharmaceutical companies, hospitals, universities and cooperative groups.

Measure	Value
PDF files in the folders	190 (3 are exact copies of others)
Unique documents	187 (168 protocols, 19 statistical analysis plans)
Total pages	19,413 (5 to 473 per document, average about 104)
Total size	about 443 MB
PDF kind	174 text, 12 mixed, 1 scanned (OCR)
Trial numbers (NCT) found	93 documents, 90 distinct trials
Processing result	187 processed, 0 failed

14.2 Dataset groups

Folder	Files / unique	Pages	Types	Notes
`1-19/`	19 / 18	1,424	11 protocols, 7 SAPs	First-in-human work and DESTINY-Breast, ASCENT, IMpassion031, KAMILLA. #10 and #11 identical.
`20-38/`	20 / 20	3,034	20 protocols	EMBRACA, monarchE, MONARCH 2, INAVO120, VERITAC-2, TROPiCS-02, IPATunity, PATHFINDER.
`39-56/`	18 / 17	2,042	17 protocols	PALOMA-3, EMERALD, ELEVATE, EMBER-3, HER2CLIMB, DESTINY-Breast04, KATHERINE. #49 and #50 identical.
`57-100/`	44 / 43	5,288	43 protocols	KATE2/3, FeDeriCa, KEYNOTE-355, ALEXANDRA, TAILORx, NSABP B-47. #58 and #59 identical.
`101-185/`	89 / 89	7,625	77 protocols, 12 SAPs	PALLAS, BRAVO, MONARCH 3, MONALEESA-3/7, PATINA, SOLE and supportive-care studies.

14.3 Content

- HER2-positive disease: trastuzumab, pertuzumab, T-DM1, T-DXd, tucatinib.
- Hormone-receptor-positive disease: palbociclib, ribociclib, abemaciclib, fulvestrant, letrozole, everolimus, alpelisib.
- Triple-negative disease: pembrolizumab, atezolizumab, sacituzumab govitecan, carboplatin, PARP inhibitors.
- Settings: neoadjuvant, adjuvant and metastatic.
- Supportive care and prevention: biosimilar filgrastim, rifaximin, elinzanetant, carvedilol, metformin, omega-3.

- Phase: Phase 2 (101), Phase 3 (55), Phase 1 (15), Phase 1b (4), Phase 3b (3), Phase 2a (3), Phase 1/2 (2), Phase 4 (1), Phase 3/4 (1), not stated (2).
- Sponsors on the title page include F. Hoffmann-La Roche (13), AstraZeneca (5), Genentech (3), International Breast Cancer Study Group (3), City of Hope (3); 94 documents state no readable sponsor.

14.4 Output dataset

Location	Content
`data/ctpip.sqlite3`	25,727 sections, 95,181 evidence chunks, 4,821 tables, 305,219 entity mentions, warnings, review items, history, audit log.
Neo4j database `ctpip`	187 Protocol nodes, 133 trials, 693 shared entities (218 drugs, 57 adverse events, 35 biomarkers, 28 endpoints, 23 conditions), 29,047 clinical relationships (28,716 validated), 175,589 relationships in total.
`data/pdfs/`, `data/artifacts/`	Stored PDFs and checkpoints of the 187 documents.

- `docs/Dataset_Catalog.csv` lists one row per document with these columns:

Column	Description
Doc No, Folder, File name	Number, containing folder and name of the PDF.
Study title	Title read from the PDF (first 220 characters).
Document type	protocol or sap.
Trial number (NCT)	Registry ID found in the text (empty if none).
Sponsor protocol code	Sponsor's own code.
Phase, Sponsor, Version, Amendment, Version date	From the title page when stated.
Pages, Size (MB), PDF kind	Length, size and text/mixed/scanned.
Entities found, Relationships in graph	Items and facts stored for the document.
Warnings	Number of reading-quality notes.

14.5 Data quality notes

Warning	Count	Meaning
`table_quality`	1,059	A table was hard to read.
`ocr_used`	195	A page was read by OCR.
`letter_spaced_text`	97	Text stored with spaces between letters.
`nct_not_found`	94	No registry number printed.
`empty_section`	42	Heading without captured text.
`sections_mostly_unknown`	14	Most headings not classified.
`eligibility_not_found`	10	No inclusion/exclusion section.
`no_headings`	3	Almost no headings detected.

- Amendments and summaries of changes are included, so some fields can legitimately be missing; CTPIP reports them as unavailable.
- The dictionary is tuned to breast cancer; other diseases need new dictionary entries.
- CTPIP organises what documents state; it is not a medical decision tool.

Appendix A. Glossary

Term	Meaning
Protocol	Document describing how a clinical trial is run.
NCT number	Public trial ID on ClinicalTrials.gov.
SAP	Statistical Analysis Plan.
Phase 1/2/3	Trial stages: safety, effectiveness signal, comparison with standard treatment.
Inclusion / exclusion criteria	Rules for who can and cannot join.
Endpoint	What a trial measures to judge success.
Entity	A recognised item: drug, disease, biomarker, endpoint.
Claim	One extracted fact with its quote and page.
Chunk	Small passage of a protocol that keeps its page and position.
Knowledge graph	Nodes (things) connected by relationships.
Neo4j / Cypher	Graph database / its query language.
OCR	Reading text from images of pages.
BFS / DFS	Breadth-first / depth-first ways of walking a graph.
Provenance	Trail from a fact to its page and sentence.
Virtual environment	Private set of Python libraries for one project.
API	Set of endpoints used by the web interface.
Local AI / LLM	A large language model that runs on this computer (through Ollama) and writes explanations.
Ollama	Free program that downloads and runs AI models locally (port 11434).
RAG	Retrieval-augmented generation: the AI first receives relevant protocol passages and then answers only from them.
Research agent	AI mode that plans several searches and comparisons before answering.
Citation / source	A numbered passage or fact ([S1]) that an AI statement is based on.
Grounding check	Automatic check of an AI answer against its sources (citations, matched statements, numbers).